

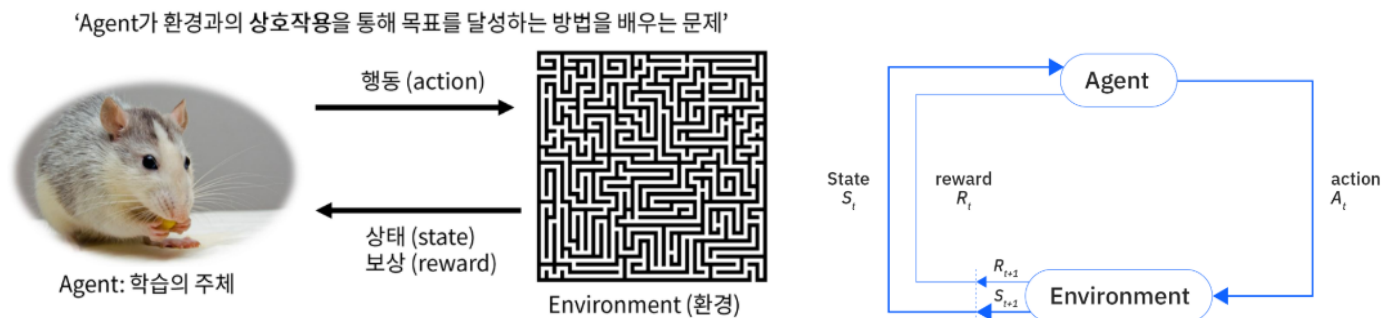
강화학습이란?

강화 학습(reinforcement learning)은 기계 학습의 한 영역이다. 행동심리학에서 영감을 받았으며, 어떤 환경 안에서 정의된 에이전트가 현재의 상태를 인식하여, 선택 가능한 행동들 중 보상을 최대화하는 행동 혹은 행동 순서를 선택한다. Agent가 Environment와 상호작용하며, Reward를 최대화하는 방향으로 Action을 선택하는 학습 방법이며, 보상을 최대화하는 방법을 학습하는 머신 러닝 기법이다. 즉, 정답 데이터 없이 환경과의 경험을 통해 최적의 행동 전략을 찾아나가는 과정이다.

이러한 문제는 매우 포괄적이기 때문에 게임 이론, 제어이론, 운용 과학, 정보이론, 시뮬레이션 기반 최적화, 다중 에이전트 시스템, 때 지능, 통계학, 유전 알고리즘 등의 분야에서도 연구된다.

● 특징

- 지도학습(Supervised Learning) 처럼 정답(Label)이 주어지지 않음
- 비지도학습(Unsupervised Learning) 처럼 데이터 간 구조를 학습하지 않음
- 오히려, "어떤 행동이 더 좋은 결과(보상)를 가져오는가?"를 **경험을 통해 학습함**



강화학습에서 입력(input)과 출력(output)은 특정 이름을 갖는다. 입력은 상태(state)라고 부른다. 출력은 AI가 수행하는 행동(action)이다. 그리고 그 사이에서는 상태를 입력으로 취하고 행동을 출력으로 반환하는 함수(처리)가 있다. 이를 정책(policy)이라고 한다.

여기에서 정책이란 에이전트가 어떤 상태(state)에서 어떤 행동(action)을 취할지를 결정하는 규칙 또는 전략이다. 수학적으로는 다음과 같이 표현된다.

$$\pi(s)=a \text{ 또는 } \pi(a|s)=P(\text{상태 } s\text{에서 행동 } a\text{를 선택할 확률})$$

"강화학습의 핵심 구성 요소 정리"

- **에이전트** : 학습 수행 주체. 환경에서 특정 행동을 선택하고 그 결과에 따라 보상을 받는다.
- **환경** : 에이전트 상호작용 대상. 에이전트의 행동에 따라 상태가 변화하고 보상이 주어진다.
- **행동** : 에이전트가 환경에서 취할 수 있는 동작.
- **보상** : 에이전트의 행동에 대한 긍정적 또는 부정적 피드백. 보상 최대화 방향으로 학습한다.
- **정책** : 에이전트가 현재 상태에서 어떤 행동을 선택할지 결정하는 규칙. 강화 학습은 최적의 정책을 학습하는 것을 목표로 한다.

강화학습의 학습 흐름

반복 구조 (에피소드 기반)

1. 환경으로부터 상태 S_t 관찰
2. 정책 π 에 따라 행동 A_t 선택
3. 행동 수행 후, 보상 R_{t+1} 및 다음 상태 S_{t+1} 관찰
4. 가치 함수 업데이트 (예: $Q(S, A)$)
5. 종료 조건 만족 시 종료, 아니면 반복

보상을 최대화하도록 정책(Policy)을 개선하면서 반복

* 강화학습의 핵심 요소 및 응용 분야

구분	내용
목적	보상을 최대화하는 행동 학습
핵심 개념	상태, 행동, 보상, 정책, 가치함수
주요 알고리즘	Q-learning, DQN, Policy Gradient, PPO 등
응용 분야	게임, 로봇, 자율주행, 금융, 의료 등
도구	OpenAI Gym, Stable-Baselines3 등

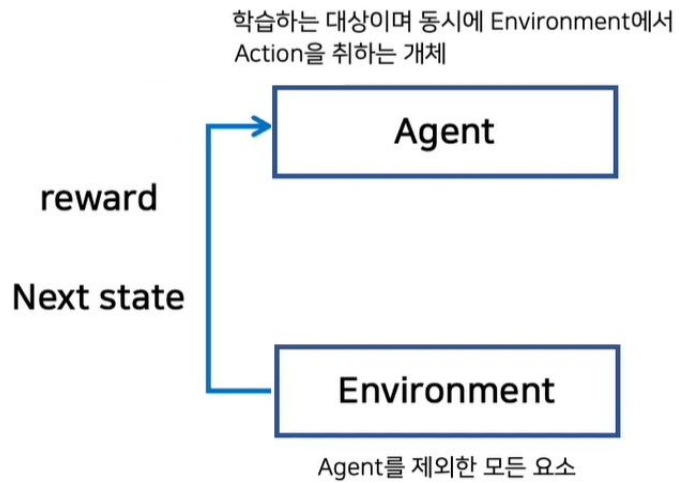
강화학습의 유형

종류	설명
Model-Free	환경의 동작 모델 없이 직접 경험으로 학습 (예: Q-learning)
Model-Based	환경 모델을 예측해 시뮬레이션하면서 학습
Value-Based	상태의 가치를 학습 (예: Q-learning, DQN)
Policy-Based	직접 정책을 최적화 (예: Policy Gradient, PPO)
Actor-Critic	정책과 가치 함수를 동시에 학습

◎ 대표적인 강화학습 알고리즘

* Q-Learning (탐색 기반) - 상태-행동 쌍의 가치를 업데이트하며 학습 - ϵ-greedy 방식으로 탐험과 이용 균형 - 탭 구조로 구성된 기본 알고리즘	* DQN (Deep Q-Network) - Q-learning + 신경망 - Atari 게임에서 사람을 능가한 성능 - 경험 Replay, 타깃 네트워크 사용
* A3C / A2C (Actor-Critic 계열) - 정책(Actor) + 가치함수(Critic)를 동시 학습 - 병렬 에이전트 학습을 통한 효율 개선	* PPO (Proximal Policy Optimization) - 안정적 성능의 Policy Gradient 계열 알고리즘 - OpenAI에서 제안, 로봇/게임 분야에 활용됨

♣ Q-러닝의 기본 구조 - 슈퍼마리오 예시



* 에이전트와 환경의 상호작용

: Agent (마리오)

- 현재 상태를 관찰하고, 점프·이동 등 행동(Action)을 선택함
- 목표: 보상(Reward)을 최대화하는 방향으로 학습

: Environment (게임 세상)

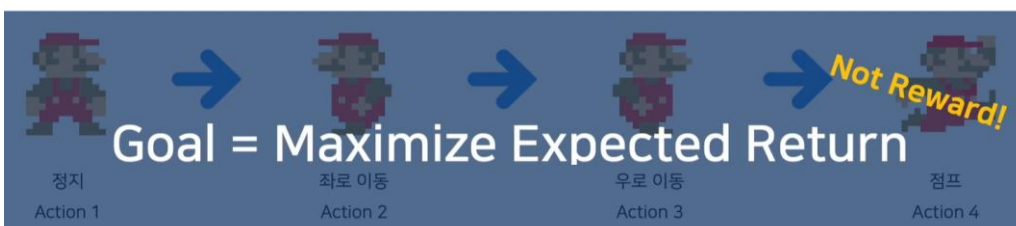
- Agent의 행동에 따라 보상(reward)과 다음 상태(next state)를 반환
- 예: 점프해서 코인을 먹으면 +1 보상, 구덩이에 빠지면 -1

* Q-러닝 학습 흐름

- 1) Agent는 현재 상태에서 행동을 선택
- 2) 환경은 그에 따른 결과(보상 + 다음 상태)를 반환
- 3) Q 테이블을 갱신하여 더 나은 행동을 선택하도록 학습

"강화학습은 마리오처럼 시도와 실패를 통해, 더 많은 점수를 얻는 법을 스스로 배워가는 과정이다!"

Sequential Decision Making Problems



Q-러닝 및 강화학습의 핵심 개념인 '순차적인 의사결정(Sequential Decision Making)'과 '보상이 아닌 기대 수익(Return)의 극대화'를 위함이다.

단순한 보상이 아닌, 기대되는 미래 보상(Return)을 고려한 의사결정

: 강화학습 에이전트(마리오)는 단 한 번의 보상이 아닌 미래까지의 누적 보상(Expected Return)을 고려하여 행동을 결정한다.

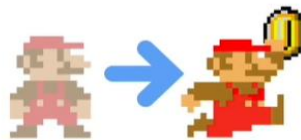
예: 지금은 점프하면 보상이 없지만, 나중에 코인을 얻을 수 있다면 점프를 선택할 수 있음!

☎ Action은 한 번의 보상보다는 "미래의 총 이득(Return)"을 최대화하는 방향으로 선택되어야 한다!

☞ Reward는 즉시 보상, Return은 미래 보상까지 포함한 총합

Expected Return

- **Reward** : 한 행동을 했을 때 받는 보상



- **Return** : 한 episode가 끝났을 때의 최종 점수



- **Expected Return**



Reward vs. Return vs. Expected Return

1) Reward

- 단일 행동에 대한 즉시 보상
- 예: 점프 후 코인 하나 먹기 → +1 보상

2) Return

- 한 에피소드 동안 누적된 총 보상
- 예: 점프 + 이동 + 점프 → 총합 점수 20점

3) Expected Return

- 앞으로 받게 될 보상의 기대 값
- 에이전트는 지금 행동이 미래에 얼마나 도움이 될지를 예측하여 판단함

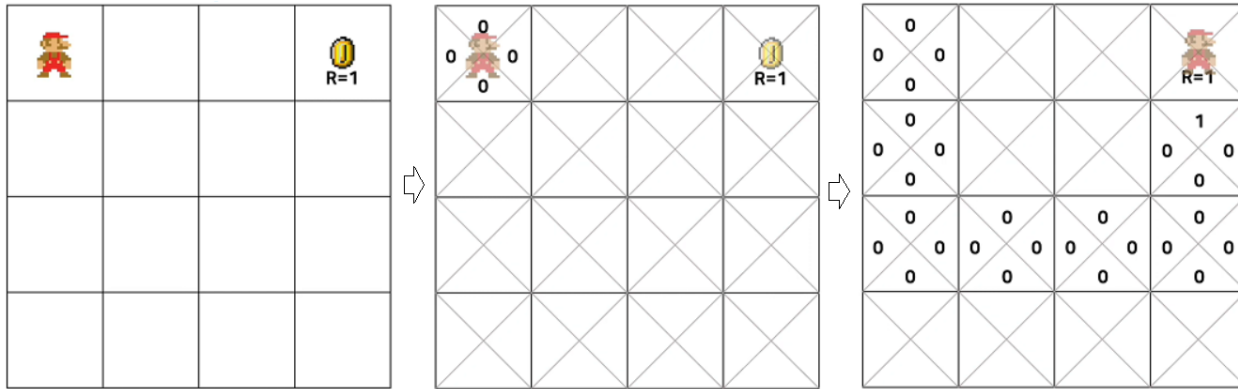
☎ 강화학습의 목표는 지금 당장 보상을 얻는 것보다, 앞으로 받을 총 보상을 예측하고 최대화하는 것!

Agent인 마리오의 경우, 지금 코인을 하나 먹느냐 보다는 앞으로 더 많은 코인을 얻기 위해 어떤 길로 가야 할지를 고민한다.

♣ Grid World는 강화학습의 핵심 개념인 Q-value의 형성과정과 Greedy한 행동 선택이 어떻게 최적의 경로로 이어지는지를 직관적으로 보여주는 환경이다.

5x3 크기의 격자 환경(Q-table)내에서 작업하는 것으로 하겠다.

Grid World Example



✓ Q-Value

- 어떤 action을 했을 때 미래에 받을 것이라고 예상되는 return의 값

✓ Greedy action

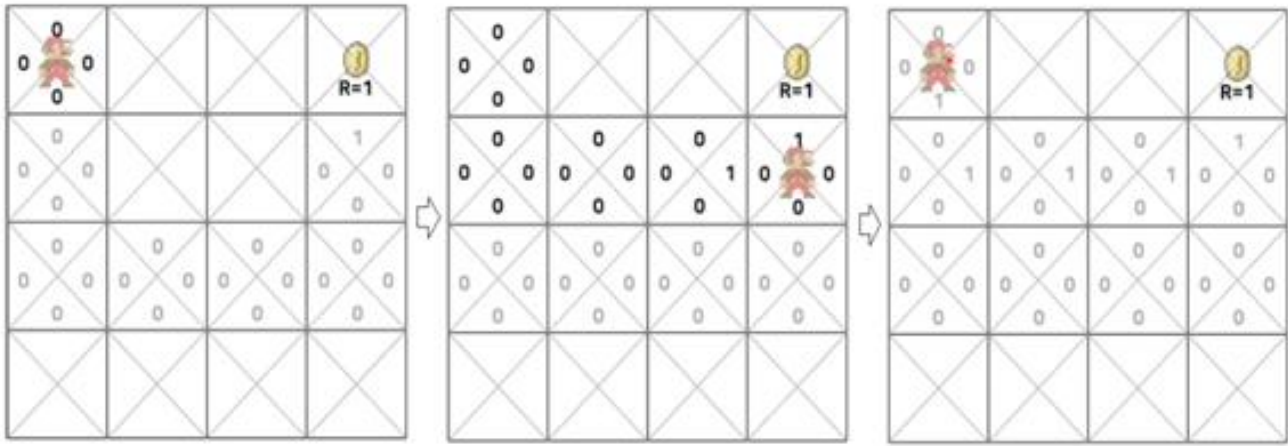
- 최적해를 구하는 데에 사용되는 근사적인 방법
- 그 순간에 최적이라고 생각되는 것을 선택해 나가는 방식으로 진행하여 최종적인 해답에 도달

agent의 목표는 가능한 행동을 선택하여 코인이 있는 칸에 도달해 최종 보상(Return)을 얻는 것이다. 초기 단계에서는 아직 환경에 대한 경험이 없기 때문에 미래의 보상(Return)에 대한 예측 값이 모두 0이다. agent의 행동에 의해 학습이 진행되면 Q-value는 변경된다.

보상 칸에는 Q값이 1로 설정되어 있으며, 그 주변 칸들에는 코인을 향해 움직이면 높은 보상을 기대할 수 있다는 정보(Q-value)가 퍼져 있다. 이러한 Q값 분포를 바탕으로 agent는 Greedy action, 즉 현재 상태에서 가장 높은 Q값을 갖는 방향의 행동을 선택하게 된다.

결과적으로 agent는 최적의 경로를 따라 목표에 도달한다.

Grid World는 강화학습의 핵심 개념인 Q-value의 형성과정과 Greedy한 행동 선택이 어떻게 최적의 경로로 이어지는지를 직관적으로 보여주는 환경이다.



반복적인 학습을 거치면서, agent는 여러 번의 에피소드를 통해 같은 경로를 반복하거나 다양한 경로를 시도하게 된다. 이 과정에서 Q값은 점차 안정적으로 갱신되며, 보상을 향한 경로에 있는 셀들의 Q값이 점차 증가한다.

결과적으로 agent가 있는 위치에서 가장 높은 Q값을 가지는 방향은 보상을 얻을 수 있는 최적의 행동으로 간주되고, 이러한 방향으로 선택을 반복하게 되면 agent는 greedy policy에 따라 효율적으로 목표 지점에 도달할 수 있게 된다. 즉, 에이전트가 행동한 정보를 바탕으로 점점 더 똑똑한 행동을 하게 된다.

더 좋은 루트가 있음에도 같은 곳만 가게 되지 않을까? Exploration



ε-greedy 정책을 사용한다. ε-greedy 정책은 0과 1 사이의 ε이라는 하이퍼파라미터를 사용하여 ε의 확률로는 무작위 행동(random action)을 선택하고, 1-ε의 확률로는 현재 가장 좋은 행동(greedy action)을 선택한다. 이러한 무작위 선택의 도입은 때때로 예상치 못한 방향으로 움직이게 만들며, 그 결과 이전에 시도하지 않았던 경로를 따라가면서 새로운 보상 경로를 발견할 가능성을 열어준다.

결과적으로 ε-greedy 정책은 에이전트가 단순히 현재 최선이라고 생각되는 경로에만 갇히지 않고, 장기적으로 더 나은 전략을 학습할 수 있도록 유도하는 중요한 메커니즘이다.

- 탐험(Exploration) : 새로운 행동을 시도하여 더 좋은 보상을 찾기
- 이용(Exploitation) : 지금까지 얻은 정보 중 가장 좋은 행동을 선택

두 개의 균형이 매우 중요함 → ε-greedy, Softmax, UCB(Upper Confidence Bound, 균형) 등 사용

이러한 구조는 Discount Factor $\gamma \in [0, 1]$ 의 성질에 기반하며, γ 의 값이 1에 가까울수록 먼 미래의 보상도 거의 동일하게 중요하게 여기고, γ 가 작을수록 당장의 보상만을 중요시하게 여긴다.

이처럼 할인율은 현재 보상과 미래 보상 사이의 중요도 균형을 조절하는 중요한 하이퍼파라미터다. Q-value 업데이트 수식은 에이전트가 어떤 상태 s_t 에서 행동 a_t 를 했을 때, 그 보상 R_t 와 다음 상태에서의 최대 Q값을 고려하여 현재 Q값을 갱신하는 방식이다:

Q-learning 알고리즘의 업데이트 식

$$Q(s_t, a_t) \leftarrow (1 - \alpha)Q(s_t, a_t) + \alpha \left(R_t + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1}) \right)$$

- $Q(s_t, a_t)$: 현재 상태 s_t 에서 행동 a_t 를 취했을 때의 Q값 (행동 가치)
- α : 학습률 (learning rate), $0 < \alpha \leq 1$
- R_t : 현재 시점에서 받은 보상 (reward)
- γ : 할인율 (discount factor), 미래 보상의 현재 가치에 대한 가중치
- $\max_{a_{t+1}} Q(s_{t+1}, a_{t+1})$: 다음 상태 s_{t+1} 에서 가능한 행동들 중 가장 높은 Q값

Q-learning의 핵심 식으로, 새로운 정보에 기반해 기존 Q값을 업데이트하는 방식

여기서 α (학습률, Learning Rate)는 현재 Q값과 새로 계산한 값 사이에서 얼마나 새 정보를 반영할지를 결정한다. $\alpha = 1$ 이면 완전히 새 값만 반영하고, $\alpha = 0$ 이면 기존 값을 그대로 유지한다.

이 수식으로 과거의 정보와 현재 경험한 보상, 미래 기대 보상을 함께 고려하여 Q값을 조금씩 보정하며 더 나은 정책(policy)을 만들어 간다.

한 마디로 할인율 γ 는 미래 보상에 대한 신뢰도, 학습률 α 는 새 경험을 얼마나 빠르게 받아들일지를 결정하며, 이 두 값은 강화학습이 얼마나 빠르게, 그리고 얼마나 멀리 내다보며 학습할지를 조절하는 열쇠 역할을 한다.

- Discount Factor (γ): 미래 보상에 얼마나 가치를 둘 것인가
- Learning Rate (α): 새 경험을 얼마나 반영할 것인가
 - 이 두 값은 Q-learning 수식의 학습 방향성과 속도를 결정한다

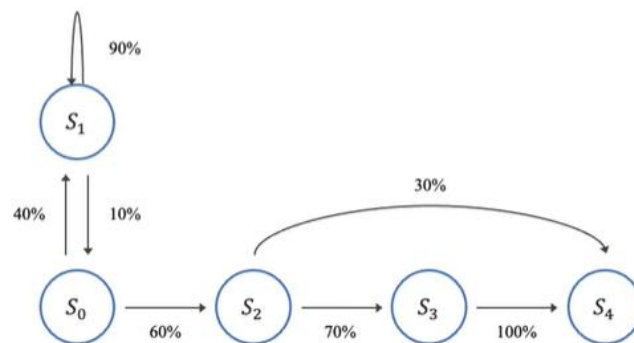
- 강화 학습에서 환경은 보통 마르코프 결정 과정(MDP, 시간에 따라 상태가 바뀌고, 확률적으로 보상이 주어지는 구조)으로 표현된다. 동적 계획법 같은 전통적인 최적화 방법은 MDP에 대한 정확한 정보가 필요하지만, 강화 학습은 그 정보를 몰라도 학습할 수 있다. 또, 강화 학습은 너무 복잡해 기존 방법으로 풀기 어려운 문제도 다룰 수 있다.

Markov property

- 과거 상태들($s_1, s_2, \dots, s_{t-1}$)과 현재 상태(s_t)가 주어졌을 때, 미래 상태(s_{t+1})는 과거 상태와는 독립적으로 현재 상태에 의해서만 결정된다는 것

✓ 전이확률(transition probability)

$$P_{SS'} = P[s_{t+1}|s_1, \dots, s_t] = P[s_{t+1} = s' | s_t = s]$$



[Markov Process 예시]

- Markov Property: 미래는 오직 현재에만 의존
 - Transition Probability: 현재 상태에서 다음 상태로 이동할 확률
- 강화학습은 이런 마르코프 구조를 기반으로 상태-행동-보상 흐름을 학습함

마르코프 성질이란, **미래의 상태는 오직 현재 상태에만 의존하며**, 과거 상태들과는 **독립적**이라는 확률적 특성을 의미한다.

수식으로 표현하면 다음과 같다 $P(s_{t+1}|s_1, s_2, \dots, s_t) = P(s_{t+1}|s_t)$

즉, 과거의 상태들이 아무리 많더라도, 지금 시점의 상태 s_t 만 알고 있다면 다음 상태 s_{t+1} 를 예측하는 데 충분하다는 것이다.

이러한 마르코프 성질을 따르는 확률 모델은 Markov Process라 불리며, MDP(Markov Decision Process) 역시 이를 전제로 구성된다.

그 아래에 제시된 다이어그램은 마르코프 프로세스의 **상태 전이 구조 예시**를 나타낸 것이다.

원으로 표현된 각 상태는 S_0, S_1, S_2, S_3, S_4 이며, 각 상태에서 다음 상태로 이동할 확률이 전이 확률(Transition Probability)로 표시되어 있다.

예를 들어 상태 S_0 에서는 60% 확률로 S_2 로 이동하고, 40% 확률로 S_1 로 이동한다.

상태 S_1 에 머물렀을 경우, 90% 확률로 제자리에 남고 10% 확률로 S_3 로 이동한 S_2 에서 S_3 로는 70%, S_4 로는 30% 확률로 전이된다 이후 S_3 는 항상(100%) S_4 로 이동한다.

이처럼 각 상태 간 이동은 오직 현재 상태에서 다음 상태로의 전이 확률만을 기반으로 하며, 이전 상태들에 대한 정보는 영향을 미치지 않는다.

이러한 마르코프 모델은 현실 세계의 많은 문제들을 단순화하여 모델링할 수 있는 장점을 가지며, 강화학습에서는 이를 바탕으로 Q-learning, DQN, Policy Gradient 등 다양한 알고리즘의 근간으로 작용한다.

Prediction & Control

Prediction

π 가 주어졌을 때 각 state의 value를 평가하는 문제

Maximize the expected return



Finding an optimal policy

Control

Optimal policy π^* 를 찾는 문제

강화학습에서 다루는 두 가지 핵심 문제인 Prediction(예측)과 Control(제어)의 개념을 구분하여 설명하고 있다.

먼저, **Prediction 문제**란 어떤 정책 π (에이전트의 행동 전략)가 주어졌을 때, 그 정책을 따르며 행동하는 동안 각 상태(state)의 가치(value)를 평가하는 문제를 말한다. 즉, 특정 정책을 가정했을 때, 각 상태가 얼마나 좋은 결과를 가져다 주는가를 추정하는 과정이다. 이는 "이 정책을 따랐을 때 얻을 수 있는 기대 보상(Maximize the expected return)"을 계산하는 것과 같다.

그 다음 단계는 **Control 문제**다. Control이란 가능한 여러 정책들 중에서 가장 좋은 정책 π 을 찾는 것을 의미한다. 즉, 기대 보상을 최대로 만들어주는 최적의 정책을 찾는 것이 목표다. 이 과정을 통해 에이전트는 Finding an optimal policy, 즉 어떤 상황에서도 가장 높은 보상을 기대할 수 있는 정책(policy)을 학습하게 된다.

이처럼 Prediction과 Control은 서로 연결되어 있다. Prediction은 주어진 정책의 가치를 평가하는 문제이고, Control은 이 예측 결과를 활용하여 최적의 정책을 찾는 문제다.

- Prediction: 주어진 정책 π 로부터 각 상태의 가치를 평가
- Control: 그 예측을 바탕으로 최적 정책 π^* 을 찾는 문제다.

이미 알고 있듯이 강화학습의 궁극적 목적은 최적의 정책을 통해 기대 보상을 최대화하는 것이다.

Value function

State value function

지금부터 기대되는 Return

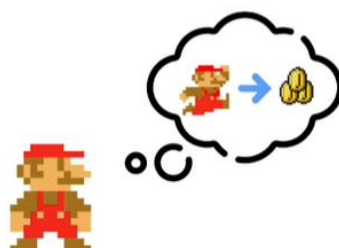
$$v_{\pi}(s_t) \triangleq \mathbb{E}_{\pi}[G_t | s_t = s]$$



Action value function

지금 행동으로부터 기대되는 Return

$$q_{\pi}(s_t, a_t) \triangleq \mathbb{E}_{\pi}[G_t | s_t = s, a_t = a]$$



강화학습에서 사용되는 두 가지 가치 함수인 상태 가치 함수(State Value Function)와 행동 가치 함수(Action Value Function)의 차이를 설명해준다.

- State Value Function은 "지금 이 상태가 얼마나 좋은가?"
- Action Value Function은 "지금 이 상태에서 이 행동을 하면 얼마나 좋은가?"

* State Value Function (상태 가치 함수)

$$v_{\pi}(s_t) = \mathbb{E}_{\pi}[G_t \mid s_t = s]$$

이 함수는 특정 상태 s_t 에 있을 때, 정책 π 를 따라 행동했을 때 앞으로 기대되는 보상의 총합(Return)을 의미한다.

예를 들어 마리오가 현재 위치에서 정해진 전략(π)에 따라 행동했을 때, 앞으로 얼마나 많은 코인을 얻을 수 있을지를 생각하고 있는 상황이다. 즉, 어떤 상태 자체가 얼마나 "좋은 상태인지"를 수치적으로 표현해주는 함수이다.

* Action Value Function (행동 가치 함수)

$$q_{\pi}(s_t, a_t) = \mathbb{E}_{\pi}[G_t \mid s_t = s, a_t = a]$$

이 함수는 특정 상태 s_t 에서 특정 행동 a_t 를 선택했을 때, 그 이후로 정책 π 를 따라 행동하며 얻을 기대 보상의 총합을 나타낸다. 즉, 현재 위치에서 특정 행동(예: 점프, 앞으로 가기)을 했을 때, 얼마나 많은 보상을 얻을 수 있을지를 계산하는 것이다. 마리오가 지금 행동을 취하면 그 행동 이후 어떤 결과가 예상되는지를 미리 생각하고 있는 모습으로 표현된다.

구분	정의	수식	의미
State Value	상태의 가치	$v_{\pi}(s)$	지금 상태에서 얻을 기대 보상
Action Value	행동의 가치	$q_{\pi}(s, a)$	지금 상태에서 특정 행동을 했을 때 얻을 기대 보상

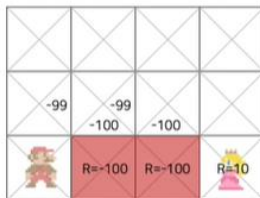
On-policy vs. Off-policy

On-policy

Target policy와 Behavior policy가 같은 경우

SARSA

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[R_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$



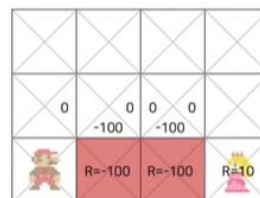
- ① 구현이 간단하다.
- ② 안전한 path를 찾는다.

Off-policy

Target policy와 Behavior policy가 다른 경우

Q-Learning

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[R_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$$



- ① 과거의 경험을 재사용할 수 있다.
- ② 사람 or other agent의 policy를 이용할 수 있다.
- ③ 일대다, 다대다 학습이 가능하다.

강화학습에서 사용하는 두 가지 주요 학습 방식인 **On-policy**와 **Off-policy**의 차이를 설명하고 있다.

이 두 방식은 정책(policy)을 어떻게 정의하고, 어떻게 학습에 사용하는지에 따라 구분된다.

- On-policy: 실제 행동한 대로 학습 (SARSA)
- Off-policy: 최적의 행동을 기준으로 학습 (Q-learning)

On-policy는 보수적이고 안전한 경로를 따르고, Off-policy는 최대 보상을 얻기 위한 도전적인 경로도 학습한다

On-policy (온-폴리시)

정의: Target policy와 Behavior policy가 같은 경우를 말한다.

즉, 학습에 사용하는 정책과 실제 행동에 사용하는 정책이 동일하다.

대표 알고리즘: **SARSA** (State-Action-Reward-State-Action)

Q값 업데이트 수식: $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [R_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$

이 수식은 실제로 행동한 a_{t+1} 에 대한 Q값을 사용하여 현재 상태-행동 쌍을 업데이트한다.

따라서 **탐험(exploration)** 정책을 그대로 반영한 학습이 이루어진다.

왼쪽에서마리오가 -99나 -100의 위험한 지역을 피하고, **안전하지만 점수가 낮은(예: R=10)** 경로를 선택하게 된다.

특징 1. 구현이 단순하고 안정적이다.

2. 실제 행동을 기반으로 학습하므로 **보수적인 학습** 경로를 따른다.

3. 안전한 경로를 찾는 데 유리하다.

Off-policy (오프-폴리시)

정의: Target policy와 Behavior policy가 다른 경우를 말한다.

즉, 다른 정책으로 수집된 경험을 바탕으로 별개의 정책을 학습할 수 있다.

대표 알고리즘: **Q-learning**

Q값 업데이트 수식: $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [R_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$

이 수식은 다음 상태에서 가능한 모든 행동 중 가장 높은 Q값(max)을 사용하여 업데이트한다.

따라서 실제로 어떤 행동을 했는지와는 **무관하게 이상적인 정책을 향해 학습**하게 된다.

오른쪽의 마리오가 -100이라는 위험을 무릅쓰더라도 **더 큰 보상(R=100)을 향한** 경로를 선택하게 된다.

즉, **이론적으로 가장 큰 보상을 주는 경로를 따라가도록** 학습된다.

특징 요약 1. 과거 경험을 재사용할 수 있어 효율적이다.

2. 사람이나 다른 agent의 정책을 기반으로 학습할 수 있다.

3. 대규모 환경에서 더 **유연하고 강력한 학습**이 가능하다.

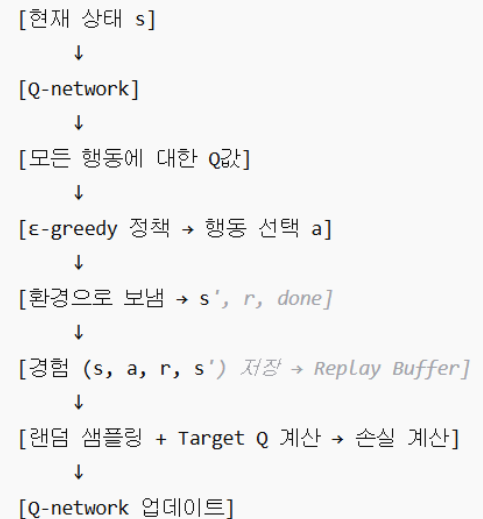
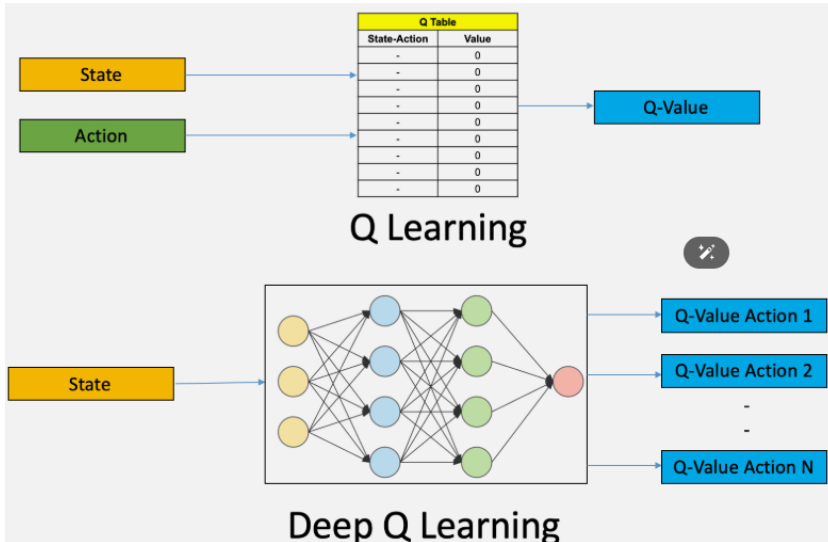
항목	On-policy	Off-policy
정책	Target = Behavior	Target ≠ Behavior
대표 알고리즘	SARSA	Q-Learning
학습 경향	실제 실행한 행동 기반 학습	최적 행동(max Q) 기준 학습
장점	안정적, 현실 반영	효율적, 재사용 가능
단점	보상 최대화가 어려울 수 있음	위험한 경로 선택 가능

DQN

딥러닝을 이용한 강화학습 알고리즘이다.

기존의 Q-learning은 Q-table(작은 표 형태)로 상태-행동 값을 저장하지만, DQN은 Q 값을 신경망(Deep Neural Network)으로 근사하여 고차원 상태 공간에서도 학습할 수 있도록 만들어졌다.

DQN 구조 다이어그램 (요약)



DQN의 장점

- 고차원 입력(예: 이미지)도 처리 가능
- 경험 재사용으로 학습 효율 증가
- 수렴 안정성 개선 (타겟 네트워크 덕분)

DQN의 한계

- 학습 속도가 느릴 수 있음
- 행동 탐색이 충분하지 않으면 지역 최적값에 머무를 수 있음
- 연속적인 행동 공간에서는 직접 사용 어려움 → DDPG 등으로 확장

DQN Loss 수식

목표값 (Target) $y = r + \gamma \cdot \max_{a'} Q_{\text{target}}(s', a')$

r: 현재 보상 γ: 할인율 (future reward의 중요도)

s': 다음 상태 Q_target: 타겟 네트워크로부터의 Q값

손실 함수 (Loss) $\mathcal{L}(\theta) = (y - Q(s, a; \theta))^2$

Q(s, a; θ): 현재 신경망(Online network)이 예측한 Q값

θ: 신경망의 파라미터

이 손실을 줄이기 위해 역전파로 θ를 업데이트

신경망은 현재 상태에서 어떤 행동을 하면 얼마나 이득일지(Q값)을

예측하려고 한다. 그런데 실제로는 $r + \gamma \cdot \max Q(s', a')$ 만큼 이득이었다

→ 이 둘의 차이를 줄여야 하므로 MSE 손실을 씀

^ 딥러닝을 강화학습에 적용할 때 몇 가지 문제가 있다:

- **Delayed Rewards:** 대부분의 딥러닝 문제들은 많은 양의 hand-labeled data를 사용하여 학습한다. 그러나 강화학습에서는 agent가 선택한 action에 대해 즉각적인 reward를 받지 않을 수 있으며, 이러한 reward는 sparse하거나 noisy하다.
- **High Correlation:** 연속적인 상태들은 서로 상관관계가 높을 수 있다. 이는 학습 데이터 간의 i.i.d 가정을 깨뜨릴 수 있다.
- **Non-stationary Distribution:** 강화학습에서는 agent가 학습을 진행함에 따라 얻게 되는 data의 distribution이 변할 수 있다. 이는 fixed underlying distribution을 가정하는 딥러닝에서 문제가 될 수 있다.

^ DQN은 이러한 문제들을 해결하기 위해 다음과 같은 기법들을 사용한다:

- Experience Replay : agent가 경험한 데이터를 저장하고 이를 무작위로 뽑아서 학습한다. 이는 데이터간의 상관관계를 줄이고, 학습 데이터의 분포를 고정시킨다.
- Fixed Q-targets : target Q-value를 계산할 때 target network를 사용하여 target Q-value를 계산한다. 이는 학습 중 target Q-value를 고정시킴으로써 학습을 안정화시킨다.

● DQN의 주요 기술 2가지

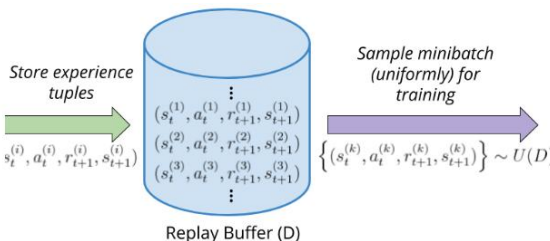
1) 리플레이 버퍼 (Replay Buffer)

→ 에이전트가 과거 경험을 저장하고, 무작위로 샘플링하여 학습.

2) 타겟 네트워크 (Target Network)

→ 안정적인 학습을 위해 Q값 업데이트에 사용되는 신경망을 고정시키고, 일정 주기로만 업데이트함.

* Replay Buffer



과거 경험을 저장하고 재사용하도록 설계된 메커니즘인 Replay Buffer는 에이전트가 환경과 상호작용하며 얻은 경험들을 저장하는 메모리 공간이다. 이 경험은 아래와 같은 튜플 형태로 저장된다.

$(s, a, r, s', done)$

s: 현재 상태(state), a: 수행한 행동(action), r: 받은 보상(reward)

s': 다음 상태(next state), done: 완료 플래그(에피소드 종료 여부)

이 데이터를 일정량 모아서 신경망 학습 시에 랜덤하게 샘플링하여 사용한다.

^^ 어떻게 작동하나?

- 매 시간 step마다 $(s, a, r, s', done)$ 형태의 경험을 버퍼에 저장
- 일정량 이상 쌓이면 랜덤하게 mini-batch 샘플을 추출
- 경사 하강 갱신 : 샘플링된 배치를 사용해 학습되고, Q 값 목표를 계산해 손실 함수 최소화.
- 버퍼가 최대 용량에 도달하면 가장 오래된 경험이 제거되어 새 경험을 위한 공간이 마련됨.
- 한 개의 에피소드가 끝났을 때의 값으로 재학습 하는 것보다 성능이 우수(논문)
- 이 샘플들을 바탕으로 DQN 모델의 Q-함수 신경망을 학습

리플레이 버퍼를 도입하면 학습 안정성과 효율성을 개선하는 여러 가지 이점이 있다.

- a) 시간적 상관관계를 끊는다.
- b) 샘플 효율성 증가.
- c) 분산을 줄이고 학습을 안정화.
- d) 정책 외 학습을 가능하게 함.

* 타겟 네트워크(Target Network)

지도학습은 데이터의 분포가 바뀌지 않으나 강화학습은 에이전트에 의해 데이터의 분포가 계속 변경된다. 이렇게 유연한 데이터는 딥러닝 모델의 학습에 좋지 않은 영향을 줄 수 있다. 타겟 네트워크는 예를 들어 1만 스텝 단위로 고장 시켜 작업하면 좀더 안정적인 학습을 하게 된다.

DQN에서는 하나의 신경망이 상태(state)를 입력받아 Q-value를 예측한다. 그런데 Q-value를 계산하고 또 그것을 업데이트하는 데 동일한 네트워크를 사용하면 학습이 매우 불안정해질 수 있다. 이를 해결하고자 Q-value를 계산하는 네트워크(메인 네트워크)와는 별도로 복사본인 '타겟 네트워크'를 사용한다.

^^ 두 개의 네트워크

- **Online Network (Q-network)** : 현재 학습 중인 네트워크. 상태로부터 Q 값을 예측하고, 파라미터를 매번 업데이트함.
- **Target Network** : 일정 주기마다 Online Network의 가중치를 복사해오는 고정된 네트워크. Q(target) 계산에 사용.

타겟 네트워크 작동 순서

1. 에이전트가 환경에서 (s, a, r, s') 경험을 얻음
2. 타겟 Q값 계산 시, **타겟 네트워크를 사용**: $y = r + \gamma \cdot \max_{a'} Q_{\text{target}}(s', a')$
3. 손실 함수 (MSE 등)를 기반으로 **Online Network를 업데이트**: $\text{loss} = (y - Q_{\text{online}}(s, a))^2$
4. 일정 주기마다 (예: 1000 스텝마다) $Q_{\text{target}} \leftarrow Q_{\text{online}}$ 으로 **파라미터 복사**